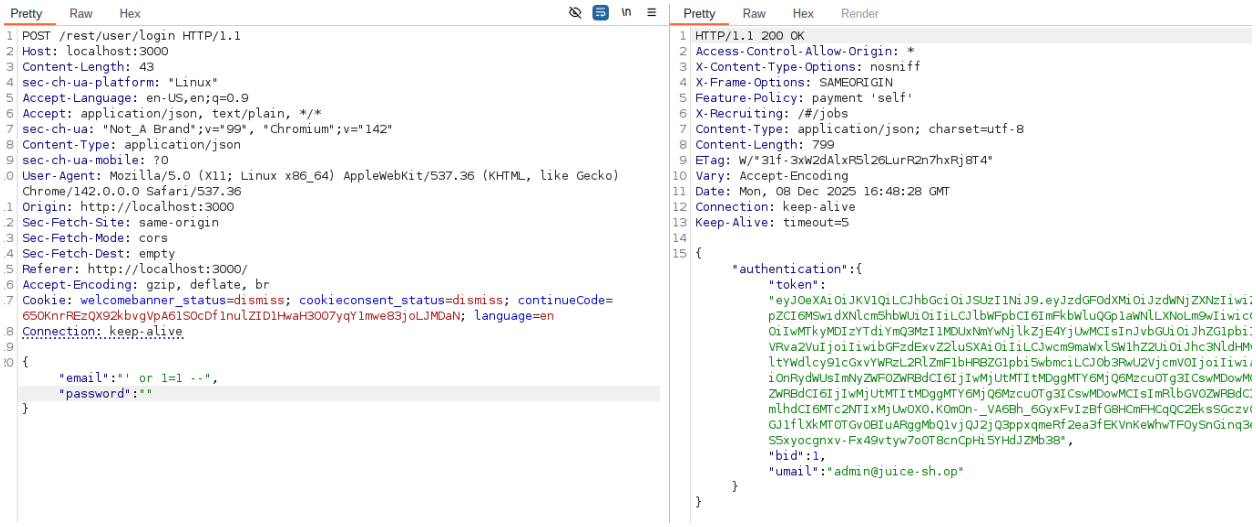
Software and mobile devices security milestone 2

First i installed docker then i fetched the Owasp juice shop from github repo then i tried to find some server-side vulnerabilities (sql injection,os (command)injection,ssrf, etc...) .

1 First bug found sql injection Login Bypass as admin:

Firstly i'll try to login normally with actual credentials then use burp suite to intercept it and then we will try Boolean Injection `' OR 1=1-- ,` `"--" is to make the rest of the query commented like checking the password or whatever.`



I logged in normally first and then i intercepted the req and sent it to the repeater and tried to add true boolean and comment the rest of the query and i have successfully did as you can see in the response it gives me 200 ok and also i have logged in as an admin , lastly to be even more sure i opened juice shop and entered " ' OR 1=1- " for the email , and any thing for the password and also i have successfully logged in as the first user which was an admin so now i have admin access and i have bypassed login perfectly .

2: SQL Injection Bug : Product Search

STEPS TO REPRODUCE:

1. Open Burp Suite and configure browser proxy to 127.0.0.1:8080

2. Navigate to http://localhost:3000 in the browser

3. In Burp Suite, go to Proxy → HTTP History

4. In browser, search for any product (e.g., "apple")

5. In Burp Suite, find the GET request:
   GET /rest/products/search?q=apple HTTP/1.1

6. Right-click the request → Send to Repeater

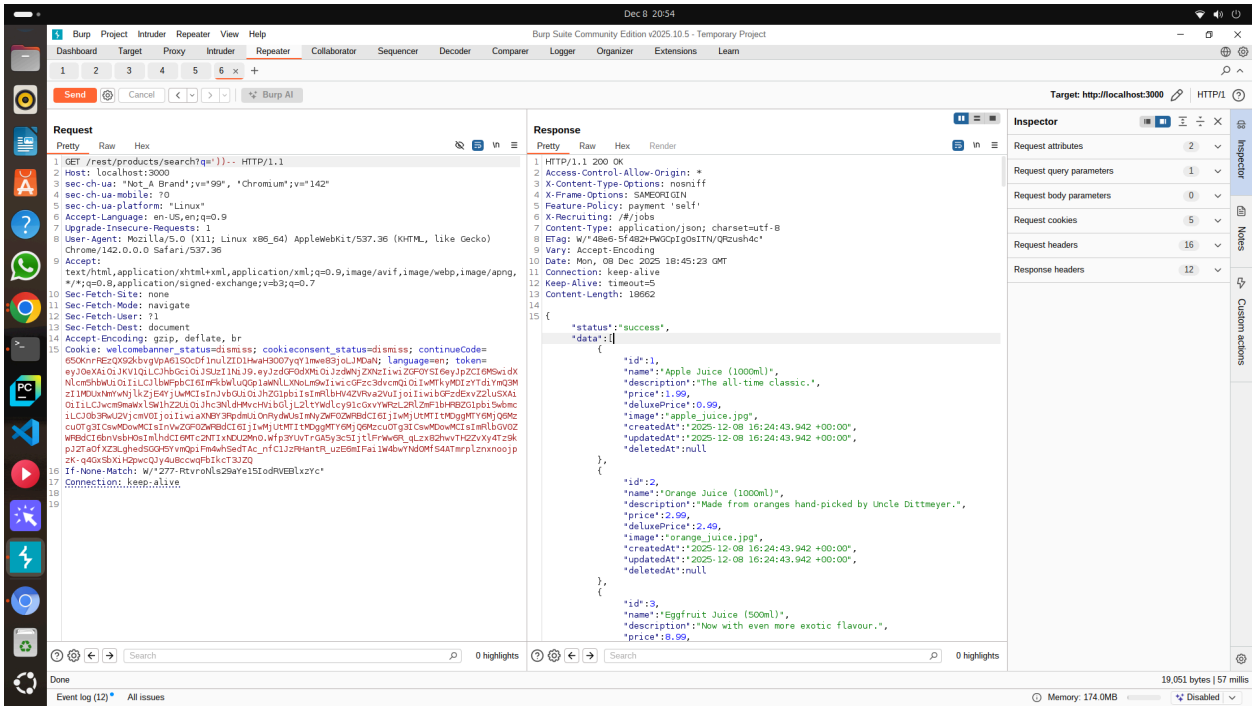
7. In Repeater tab, modify the request to:
   GET /rest/products/search?q='))-- HTTP/1.1

8. Click "Send" button

9. Observe the response in the Response tab
This gets all the products even if they are deleted because '))--

Original SQL Query (estimated): SELECT * FROM Products WHERE ((name LIKE '%SEARCH%' OR description LIKE '%SEARCH%')) AND deletedAt IS NULL

Exploited SQL Query: SELECT * FROM Products WHERE ((name LIKE '%'))--' OR description LIKE '%'))-- AND deletedAt IS NULL The -- comments out everything after it, removing the deletedAt filter.



This was an example of a deleted product : "id":10,"name":"Christmas Super-Surprise-Box (2014 Edition)","description":"Contains a random selection of 10 bottles (each 500ml) of our tastiest juices and an extra fan shirt for an unbeatable price! (Seasonal special offer! Limited availability!)","price":29.99,"deluxePrice":29.99,"image":"undefined.jpg","createdAt":"2025-12-08 16:24:43.945 +00:00","updatedAt":"2025-12-08 16:24:43.945 +00:00","deletedAt":"2025-12-08 16:24:44.056 +00:00"

The output is pretty long so i will not paste it all here.

Sql mittgations :

1• Use prepared statements / parameterized queries

Prevents user input from being executed as SQL.

```
const users = await db.query(
  "SELECT * FROM Users WHERE email = ?",
  { replacements: [email] }
);
```

2• Server-side input validation

Accept only the expected data type or format.

Example:

```
if (!/^[a-zA-Z0-9 ]+$/.test(searchTerm)) {
  return res.status(400).send("Invalid input");
}
```

3• Apply least-privilege database accounts

Use DB users with minimal permissions.

Example:
Create a DB user that can only:

- SELECT
- INSERT
- UPDATE
  —not DROP or DELETE.

4• Disable detailed error messages

Show generic user errors; log full errors internally.

Example:

```
User message: "An error occurred"
Log message: "[SQL ERROR] Syntax error at SELECT..."
```

5. Blacklist (Not recommended alone)

Block known bad characters or patterns.

✗ Example (Blacklist)

```
if (searchTerm.includes("'") || searchTerm.includes("--")) {
  return res.status(400).send("Invalid input");
}
```

Why blacklist is weak

Attackers can simply:

- use alternate encodings
- use bypass characters
- use comment tricks
- use whitespace variations

Blacklists **never cover all payload variations**.

6.orm:Use ORMs that escape values automatically

Object-Relational Mapping (ORM) frameworks help prevent SQL Injection by allowing developers to interact with the database using safe methods and objects instead of writing raw SQL queries. ORMs automatically escape and parameterize user inputs, ensuring that data is treated strictly as values, not executable SQL code. This significantly reduces the risk of injection vulnerabilities caused by string concatenation. For example, using ORM methods like `findOne()` or `findAll()` (Sequelize), `session.query()` (SQLAlchemy), or `session.get()` (Hibernate) generates secure, parameterized SQL under the hood. While ORMs improve security, they must be used correctly—developers should avoid raw SQL inside ORM functions and still apply server-side validation. Overall, ORMs greatly reduce the likelihood of SQL Injection by enforcing safe query patterns.

3 ROKEN ACCESS CONTROL VULNERABILITY

Vulnerability Name: Insecure Direct Object Reference (IDOR) in Basket API,

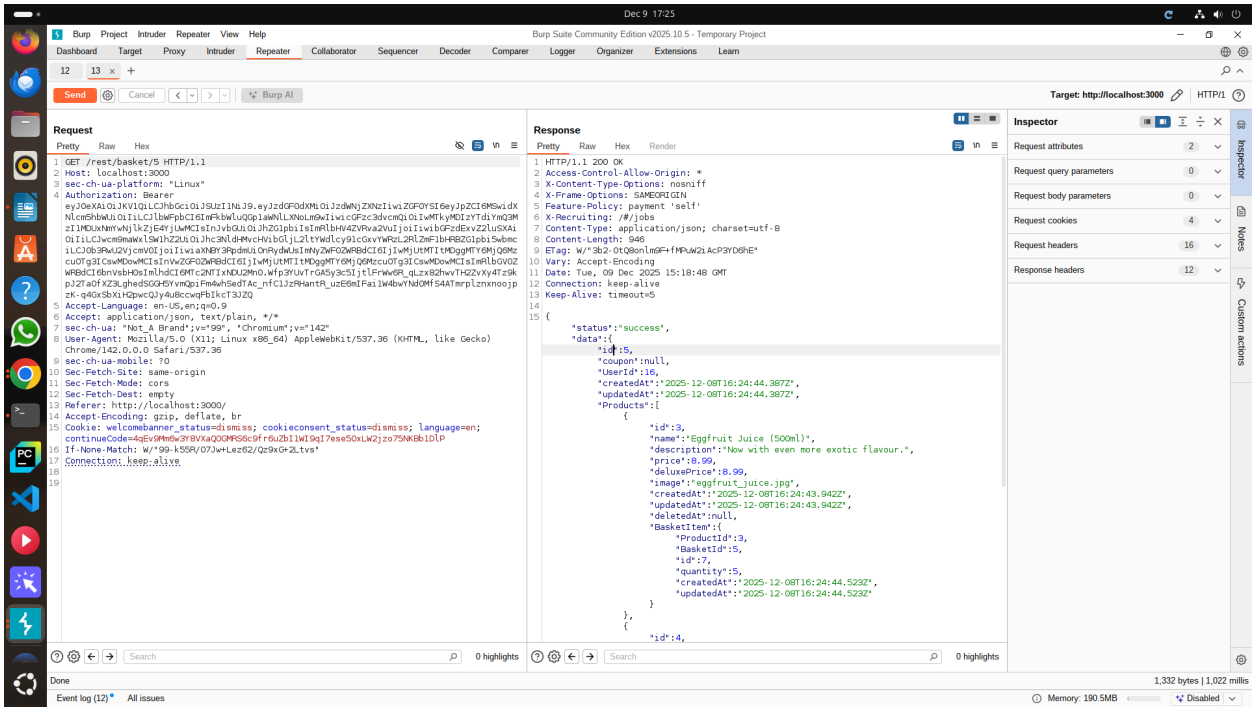
DESCRIPTION: The shopping basket API endpoint /rest/basket/{basketId} does not properly validate that the authenticated user has permission to access the requested basket. By simply changing the basket ID parameter in the URL, any authenticated user can view the contents of any other user's shopping basket, including product details, quantities, and user information.

STEPS TO REPRODUCE:
1. Open Burp Suite and configure browser proxy to 127.0.0.1:8080
2. Log in to the Juice Shop application at http://localhost:3000
3. Add any product to your shopping basket by clicking "Add to Basket"
4. In Burp Suite → Proxy → HTTP History, locate the GET request: GET /rest/basket/{YOUR_BASKET_ID} HTTP/1.1
5. Right-click the request → Send to Repeater (Ctrl+R)
6. Go to Repeater tab
7. Modify the basket ID in the request from your ID to another number: Original: GET /rest/basket/2 HTTP/1.1 Modified: GET /rest/basket/5 HTTP/1.1
8. Click "Send" button
9. Observe the Response panel shows HTTP 200 OK with another user's basket contents

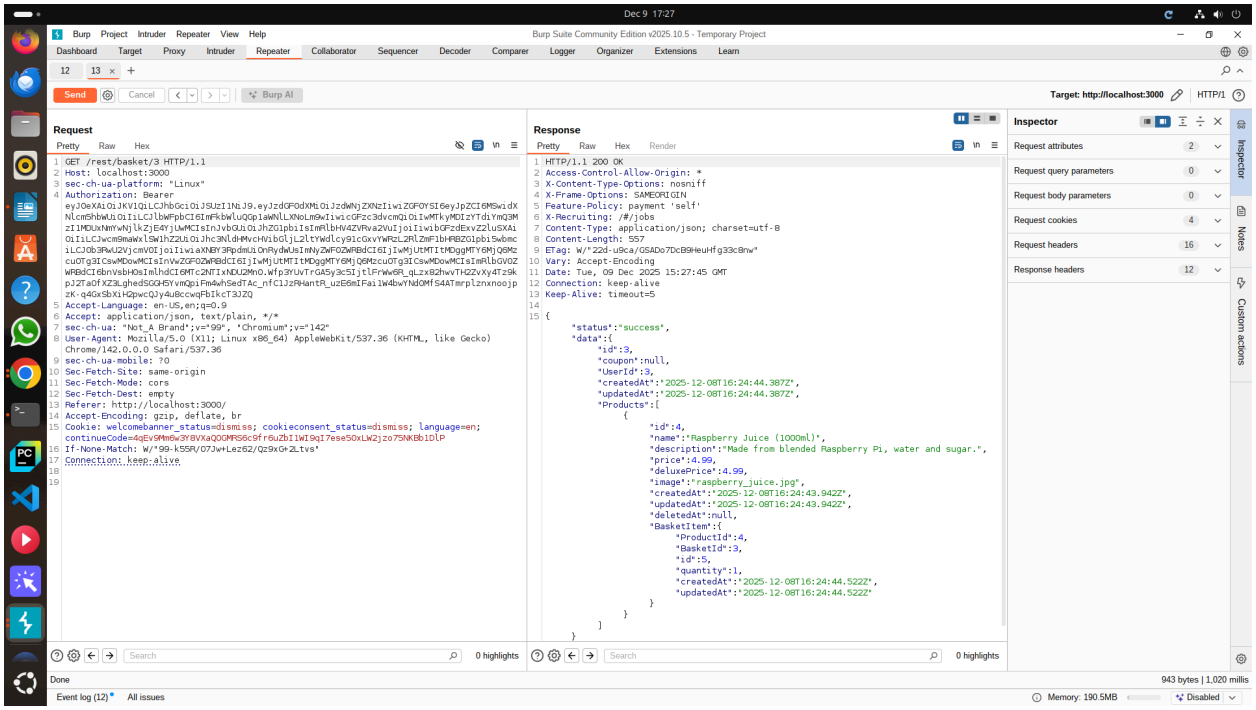
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 946
ETag: W/"3b2-0tQ8onlm9F+fMPuW2iAcP3YD6hE"
Vary: Accept-Encoding
Date: Tue, 09 Dec 2025 15:18:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":{"id":5,"coupon":null,"UserId":16,"createdAt":"2025-12-08T16:24:44.387Z","updatedAt":"2025-12-08T16:24:44.387Z","Products":[{"id":3,"name":"Eggfruit Juice (500ml)","description":"Now with even more exotic flavour.","price":8.99,"deluxePrice":8.99,"image":"eggfruit_juice.jpg","createdAt":"2025-12-08T16:24:43.942Z","updatedAt":"2025-12-08T16:24:43.942Z","deletedAt":null,"BasketItem":{"ProductI

d":3,"BasketId":5,"id":7,"quantity":5,"createdAt":"2025-12-08T16:24:44.523Z","updatedAt":"2025-12-08T16:24:44.523Z"}},{"id":4,"name":"Raspberry Juice (1000ml)","description":"Made from blended Raspberry Pi, water and sugar.","price":4.99,"deluxePrice":4.99,"image":"raspberry_juice.jpg","createdAt":"2025-12-08T16:24:43.942Z","updatedAt":"2025-12-08T16:24:43.942Z","deletedAt":null,"BasketItem":{"ProductId":4,"BasketId":5,"id":8,"quantity":2,"createdAt":"2025-12-08T16:24:44.523Z","updatedAt":"2025-12-08T16:24:44.523Z"}}]}}



I also changed it to id 3 and 2 and 6 to observe the changes



IDOR Mitigations :

1. Implement Server-Side Authorization Checks

Always verify that the authenticated user owns the requested object (e.g., basket).
Without this check, attackers can access other users' data by changing IDs.

---

2. Use Indirect References (UUIDs instead of numeric IDs)

Replace predictable IDs like `1`, `2`, `3` with UUIDs (`a38f-92bd-...`).
This prevents easy guessing or enumeration of object identifiers.

---

3. Enforce "Object Belongs to User" Validation

Before returning data, verify:
`if (object.userId !== loggedInUser.id) return 403.`
Stops attackers from accessing objects they don't own.

---

4. Remove Sensitive IDs From URLs

Use endpoints like `/rest/basket/mine` instead of `/rest/basket/{id}`.
Eliminates object IDs from attacker control.

---

5. Add Access Control Middleware

Implement a centralized authorization layer (RBAC/ABAC).
Prevents developers from forgetting checks in individual routes.

---

6. Add Logging for Unauthorized Access Attempts

Log every access where `userId != ownerId`.
Helps detect IDOR attempts early.

---

7. Rate Limit ID-Based Endpoints

Limit how many `/rest/basket/{id}` requests a user can make per minute.
Reduces the ability to brute-force or enumerate IDs.

---

8. Perform Security Testing (Automated + Manual)

Add IDOR tests to CI/CD and run regular pentests.
Prevents regressions and finds new insecure endpoints.

---

9. Educate Developers on Broken Access Control

Train developers on OWASP Top 10 and secure coding.
Reduces the chance of introducing new IDORs.

---

4 Broken Access Control - Privilege Escalation Through Manipulated Registration Request

2. Description

OWASP Juice Shop contains a server-side access control flaw in the user registration endpoint. During account creation, the backend blindly trusts the role field sent by the client, allowing a normal user to assign themselves administrative privileges.

By intercepting the registration request using Burp Suite and injecting "role": "admin" (or URL-encoded equivalent), an attacker can create a full administrator account without legitimate credentials.

This vulnerability demonstrates a critical failure in server-side authorization logic.

———

3. Impact

If exploited, an attacker can:

- Create an unauthorized administrator account
- Gain full access to the admin panel
- View, modify, or delete products
- Access all user accounts
- Retrieve sensitive information
- Completely compromise the application
- Perform destructive administrative actions

Business Impact: Full application takeover

steps to Reproduce (Short Version)

1. Open Registration Page
   - Go to: Account → Login → Not yet a customer (Register).
2. Fill Registration Form
   - Email: attackerAdmin@example.com
   - Password: Test1234!
   - Repeat Password: Test1234!
   - Any security question + answer.
   - Turn Burp Intercept ON before clicking Register.
3. Intercept the Request in Burp

- Look for:

POST /api/Users

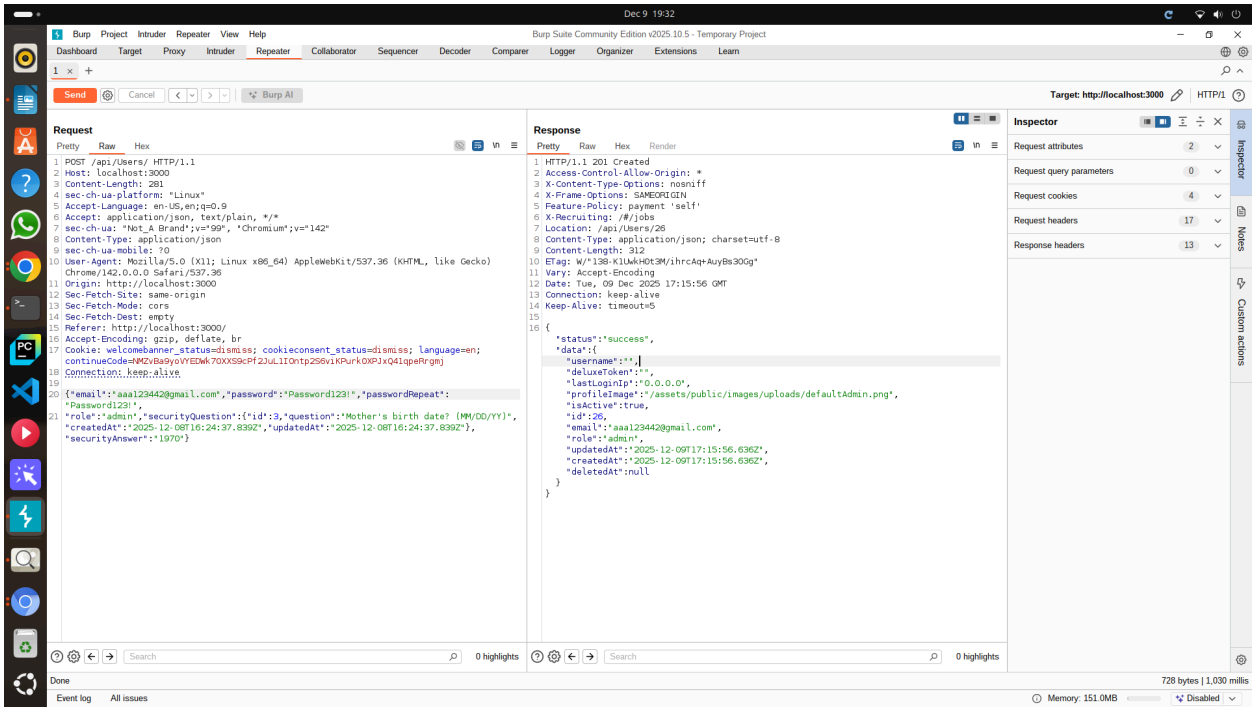
- The body will be either:
- JSON – {"email": "...", "password": "...", ...}
- or URL-encoded – email=...&password=...&...

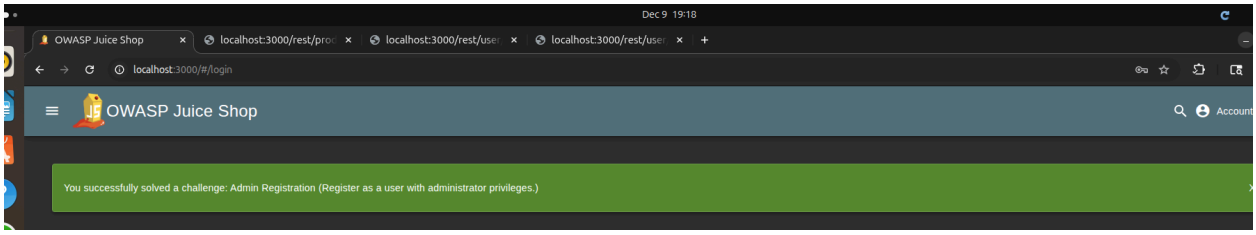
4. Add role=admin to the Request
- If JSON body, modify to include:

```
{
  "email": "attackerAdmin@example.com",
  "password": "Test1234!",
  "passwordRepeat": "Test1234!",
  "role": "admin",
  "securityAnswer": "test"
}
```

5. Forward the Request
- Click Forward (or Send if using Repeater).
- The response should show:

6. Log In and Confirm Admin Access
- Login with:
- Email: attackerAdmin@example.com
- Password: Test1234!
- Go to /#/administration and confirm you can access the admin panel.





Mittigations:

1. Enforce Server-Side Role Assignment
   - Do not accept a role field from the client in registration requests.
   - Assign all new users a default role (e.g., "customer") on the backend only.
   - Reject or strip any unexpected fields from client input.

---

2. Implement Strict Input Validation
   - Allow only approved fields (email, password, securityAnswer, etc.).
   - Reject requests containing additional parameters like role.
   - Validate and sanitize all incoming JSON or form data.

---

3. Add Authorization Checks
   - Only authenticated admins should be able to create or update admin accounts.
   - Protect role-changing operations with server-side permission checks.
   - Enforce the Principle of Least Privilege.

---

4. Secure JWT Handling
   - Ensure JWT tokens are signed and verified for every request.
   - Reject tokens with manipulated payloads such as altered roles.
   - Use short-lived tokens and proper signature algorithms.

---

5. Harden API Endpoints

- Use a strict schema validation library (e.g., JOI, Yup, Marshmallow).
- Reject requests that do not match the expected schema.
- Remove sensitive fields from public API responses.

---

6. Add Logging and Monitoring

- Log all registration attempts with unexpected parameters.
- Alert when multiple admin accounts are created.
- Monitor misuse of admin-only actions.

---

7. Conduct Regular Security Testing

- Perform periodic access control tests.
- Add unit and integration tests preventing privilege escalation.
- Include Broken Access Control in automated security scans.

---

8. Follow Secure Development Best Practices

- Adopt a Zero-Trust approach toward client-provided data.
- Review and audit API endpoints during development.
- Educate developers about OWASP Top 10 issues.

5th Vulnerability: Missing Rate Limiting on Login Endpoint

Description

The Juice Shop application does not enforce rate limiting or account lockout on the /rest/user/login endpoint. This allows an attacker to perform unlimited authentication attempts without any delay, captcha, or blocking mechanism. As a result, an attacker can brute-force user credentials using automated tools.

⭐ Steps to Reproduce

1. Intercept a login request using Burp Suite.
2. Send the request to Intruder.
3. Highlight the password field as the payload position.
4. Use a simple wordlist (e.g., 123456, admin, password).
5. Start the attack.
6. Observe that the server accepts unlimited requests without blocking or slowing down the attacker.
7. If one of the tested passwords is correct, the server returns: {"status":"success"}

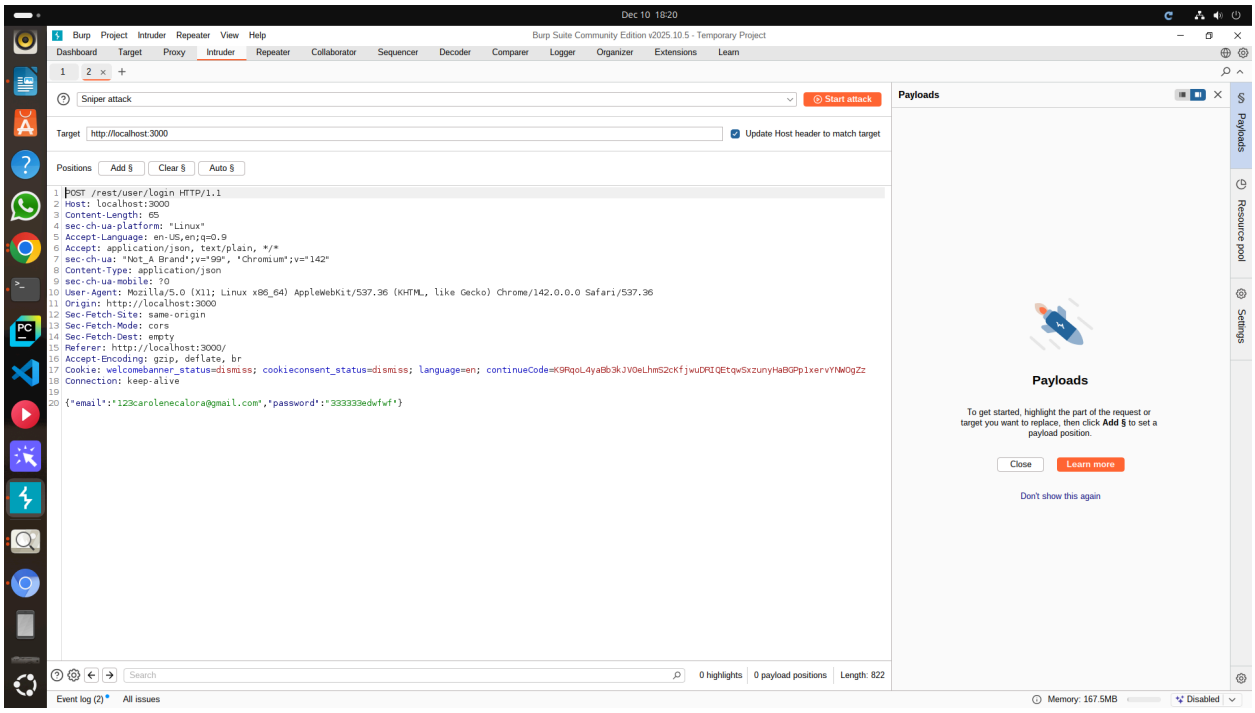
⭐ Impact

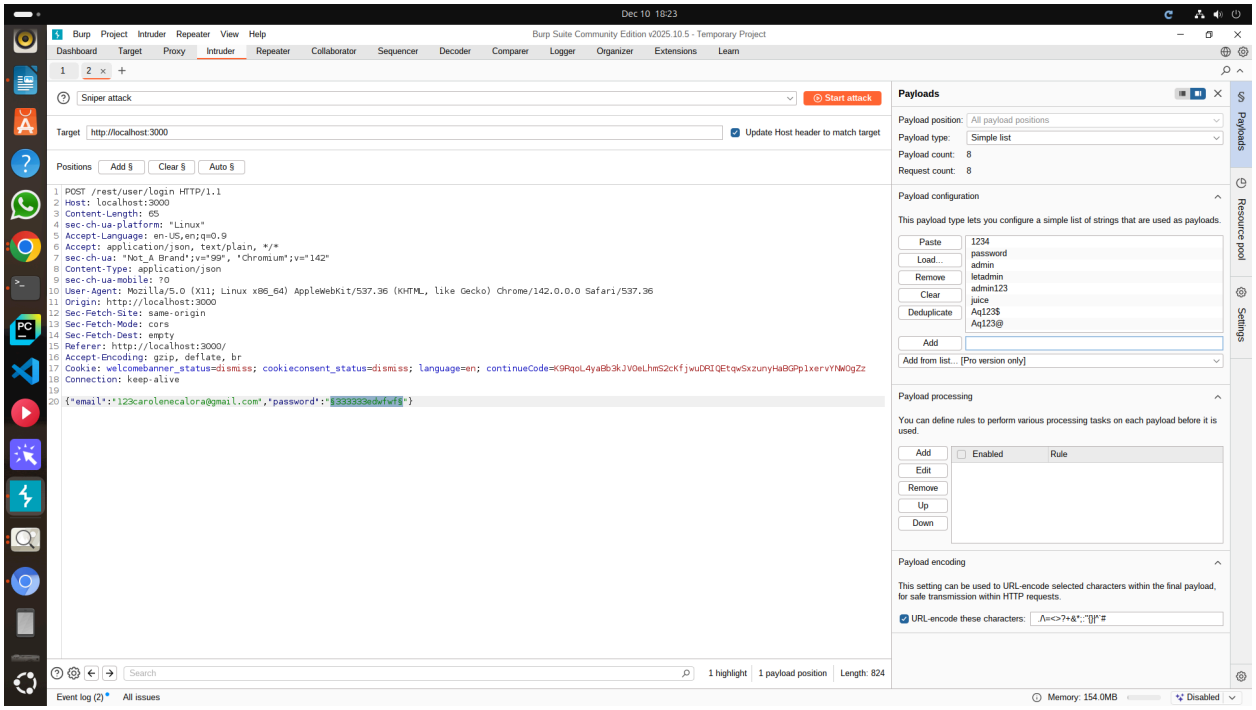
- Attackers can brute-force user accounts
- Admin accounts can be discovered and compromised
- No protection against credential stuffing
- Can lead to full account takeover
- Enables other attacks such as password reset abuse

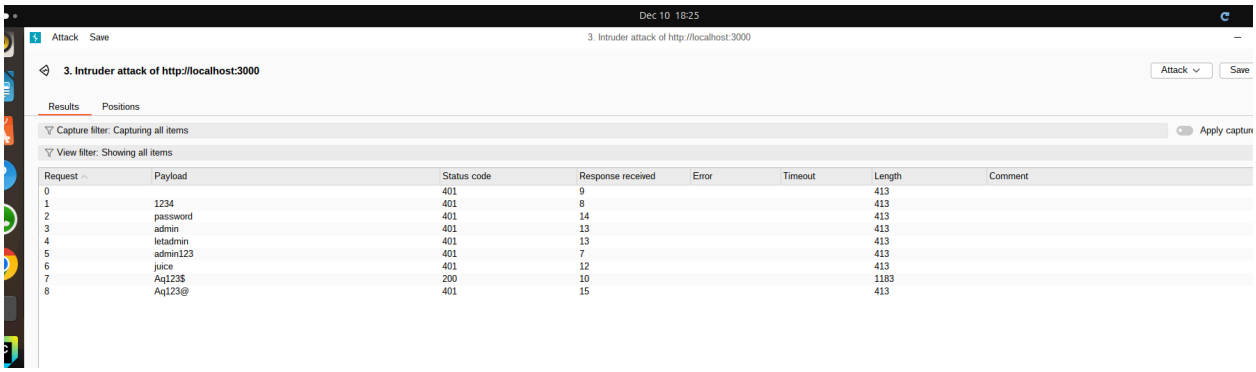
If they already knew the email.

⭐ Mitigation

- Implement rate limiting (e.g., max 5 attempts per minute)
- Add CAPTCHA after several failed attempts
- Enforce temporary account lockout
- Add MFA
- Monitor excessive login attempts
- Use tools such as fail2ban or IP throttling







As we can see here password Aq123$ gives status code 200 and the length i way more than the others now we can have full control of the user account .

⭐ Mitigation

1. Implement Rate Limiting

Limit the number of login attempts per minute to prevent automated brute-force attacks.

2. Add Account Lockout After Several Failed Attempts

Temporarily lock the account after too many wrong passwords to stop continuous guessing.

3. Use CAPTCHA After Multiple Failed Logins

A CAPTCHA forces human verification and blocks automated attackers.

4. Enforce Strong Password Policies

Requiring long and complex passwords reduces the chance of successful brute-force attempts.

5. Add Multi-Factor Authentication (MFA)

Even if a password is guessed, attackers cannot log in without the second factor.

6. Monitor and Log Failed Login Attempts

Detecting unusual patterns of failed logins helps identify and stop attacks early.

7. Block or Throttle Suspicious IP Addresses

Automatically slow down or block IPs that send too many login requests.

8. Introduce Password Attempt Cool-down Delays

Adding a small delay after each failed login reduces the effectiveness of rapid brute-force.

9. Use Password Hashing and Salting

Storing passwords securely prevents attackers from exploiting leaked or weak credentials.

10. Implement Device Fingerprinting or Login Anomaly Detection

Detects unusual login behavior (new device, location, rapid attempts) and challenges the user.

Not 100 percent verefied

6thVulnerability: Authentication Bypass via JWT "None Algorithm" Forgery

Description

The application accepts JWT tokens without validating their signature when the algorithm is set to "none". This allows an attacker to forge an unsigned token, modify the user role to "admin", and gain full administrative access. This is a server-side authentication failure.

Steps to Reproduce (Burp Suite Only)

1. Capture JWT

1. Log in normally.
2. Burp → Proxy → Intercept ON.
3. Refresh the page to capture a request

4. Send this request to Repeater.

2. Decode JWT in Burp

1. Copy the token → Burp Decoder.
2. Base64URL-decode the header & payload.

Original Header

{"typ":"JWT","alg":"RS256"}

Original Payload:

{"data":{"role":"customer"}}

3. Modify Header + Payload

Change header:

{"typ":"JWT","alg":"none"}

Change payload:

{"data":{"id":1,"email":"admin@juice-sh.op","role":"admin"}}

4. Re-encode

1. Base64URL-encode the modified header and payload.
2. Build a new token:

<encoded_header>.<encoded_payload>.

5. Use Forged Token

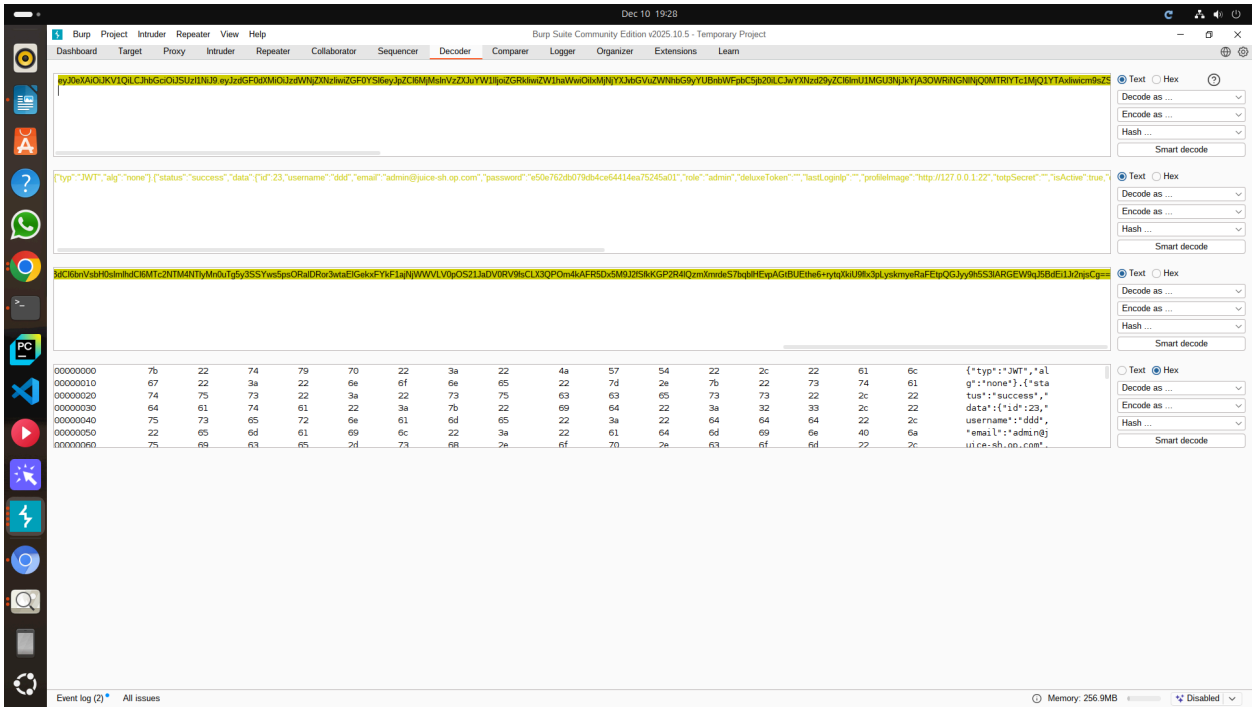
Replace the Authorization header in Repeater

Send the request → the response should shows "role": "admin".

You now have admin access.

Impact

- Full admin takeover
- Total authentication bypass
- Ability to modify all data
- Complete compromise of user accounts



Mitigations:

1. Reject tokens using alg: none

Never allow unsigned JWTs, as this directly enables token forgery.

2. Enforce strict server-side signature verification

Always verify the signature using the expected algorithm before trusting the token.

3. Use strong cryptographic algorithms (HS256 or RS256)

Secure signing algorithms prevent attackers from creating valid fake tokens.

4. Store the JWT secret/private key securely

Protecting the key prevents attackers from re-signing tokens with valid signatures.

5. Implement short token expiration times

Limits the window for attackers to reuse stolen or forged tokens.

6. Validate user roles and IDs on the server

Do not rely solely on token claims; cross-check against the database.

7. Rotate and invalidate old JWT keys regularly

If a secret is leaked or misused, rotating keys prevents long-term exploitation.

8. Use a trusted JWT library with unsafe algorithms disabled

Ensures the application does not accidentally allow insecure algorithms.

9. Log and monitor failed or suspicious token validations

Helps detect brute forcing or tampering attempts early.

Note :you can use :https://www.jwt.io/ to decode nd encode and ,etc...;